

PATRÓN DE DISEÑO: OBSERVADOR (OBSERVER)

Capítulo 33 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El patrón Observador es útil para diseñar un modelo de comunicación consistente entre un conjunto de objetos dependientes y el objeto del que dependen. Esto permite que los objetos dependientes mantengan su estado sincronizado con el objeto del que dependen. El conjunto de objetos dependientes se denomina observadores, y el objeto del que dependen se denomina sujeto.

Para lograr esto, el patrón Observador propone un modelo de publicación-suscripción que establece una frontera clara entre el conjunto de objetos Observador y el objeto Sujeto.

Un observador típico es un objeto con interés o dependencia en el estado del sujeto. Un sujeto puede tener más de un observador de este tipo. Cada uno de los observadores necesita conocer cuándo el sujeto experimenta un cambio en su estado.

El sujeto no puede mantener una lista estática de dichos observadores, ya que la lista de observadores de un sujeto puede cambiar dinámicamente. Por ello, cualquier objeto interesado en el estado del sujeto debe registrarse explícitamente como observador ante el sujeto. Cada vez que el sujeto experimenta un cambio en su estado, notifica a todos los observadores registrados. Al recibir la notificación del sujeto, cada uno de los observadores consulta al sujeto para sincronizar su estado con el de este. De este modo, el sujeto actúa como publicador al enviar mensajes a los observadores suscritos.

En otras palabras, el escenario contiene una relación de uno a muchos entre el sujeto y el conjunto de sus observadores. Cada vez que la instancia del sujeto experimenta un cambio de estado, todos sus observadores dependientes son notificados y se actualizan. Cada objeto observador debe registrarse con el sujeto para ser notificado cuando haya un cambio en el estado del sujeto. Un observador puede registrarse o suscribirse a múltiples sujetos. Cuando un observador ya no desea ser notificado, se da de baja del sujeto.

Para que este mecanismo funcione:

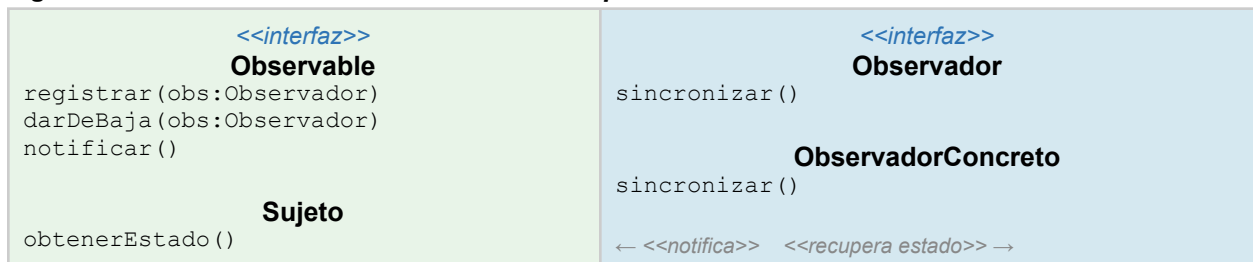
- El sujeto debe proporcionar una interfaz para registrar y dar de baja observadores que deseen recibir notificaciones de cambio.
- Una de las siguientes dos condiciones debe cumplirse:
 - En el modelo de extracción (pull model) — El sujeto debe proporcionar una interfaz que permita a los observadores consultarle el estado requerido para actualizar su propio estado.
 - En el modelo de empuje (push model) — El sujeto debe enviar la información de estado que los observadores puedan necesitar.
- Los observadores deben proporcionar una interfaz para recibir notificaciones del sujeto.

El diagrama de clases de la Figura 1 describe la estructura de las diferentes clases y su asociación, atendiendo a los requisitos enumerados anteriormente. De este diagrama de clases se puede observar que:

- Se espera que todos los sujetos proporcionen implementación para una interfaz equivalente a la interfaz Observable.
- Se espera que todos los observadores cuenten con una interfaz similar a la interfaz Observer.

Pueden considerarse diversas variantes al aplicar el patrón Observador según los diferentes tipos de relaciones sujeto-observador, como observadores interesados únicamente en ciertos tipos de cambios del sujeto.

Figura 1 — Asociación Genérica de Clases al Aplicar el Patrón Observador



Agregar Nuevos Observadores

Tras aplicar el patrón Observador, es posible agregar diferentes observadores sin necesidad de realizar ningún cambio en la clase Sujeto. Del mismo modo, los observadores no se ven afectados cuando cambia la lógica de cambio de estado del sujeto.

2. Ejemplo

Construyamos una aplicación de informes de ventas para la gestión de una empresa con múltiples departamentos. Las funcionalidades de la aplicación incluyen:

- Los usuarios deben poder seleccionar un departamento específico de su interés.
- Al seleccionar un departamento, deben generarse dos tipos de informes:
 - Informe mensual — Una lista de todas las transacciones del mes actual para el departamento seleccionado.
 - Gráfico de ventas acumuladas anuales (GAA) — Un gráfico que muestra las ventas acumuladas del año para ese departamento, organizadas por mes.
- Cada vez que se selecciona un departamento diferente, ambos informes deben actualizarse con los datos del departamento actualmente seleccionado.

A partir de la funcionalidad propuesta, podemos identificar fácilmente que ambos objetos de reporte dependen del objeto que mantiene el departamento seleccionado por el usuario. Podemos aplicar el patrón Observador para establecer un modelo de comunicación consistente entre el objeto que gestiona la selección del usuario y ambos objetos de reporte dependientes.

Definamos tres clases con la funcionalidad indicada, tal como se muestra en la Tabla 1. Aplicando el patrón Observador, definamos una interfaz Observable a ser implementada por el GestorReportes (Figura 2).

Tabla 1 — Clases Sujeto-Observador

Clase	Rol	Funcionalidad
GestorReportes	Sujeto	Muestra la interfaz de usuario para que el usuario seleccione un departamento. Mantiene el departamento seleccionado en una variable de instancia.
InformeMensual	Observador	Muestra el informe mensual del departamento seleccionado.
GraficoGAA	Observador	Muestra el gráfico de ventas acumuladas anuales (GAA) del departamento seleccionado.

Interfaz Observable

```
public interface Observable {
    public void notificarObservadores();
    public void registrar(Observador obs);
    public void darDeBaja(Observador obs);
}
```

La clase GestorReportes (Listado 1) provee la implementación de los métodos declarados en la interfaz Observable. Ambos objetos de reporte dependientes utilizan estos métodos para registrarse como observadores. El GestorReportes almacena cada uno de estos observadores registrados en la variable listaObservadores. El departamento actualmente seleccionado constituye el estado del GestorReportes y se mantiene en una variable de instancia denominada departamento. Cada vez que se establece un nuevo valor para la variable departamento (lo que constituye un cambio de estado), se invoca el método notificarObservadores. Al ejecutarse, el GestorReportes invoca el método actualizarDatos(Observable) sobre cada uno de sus observadores registrados en ese momento.

Además de proporcionar la implementación de la interfaz Observable, el GestorReportes muestra la interfaz de usuario necesaria (Figura 3) para permitir al usuario seleccionar un departamento de interés.

Definamos también una interfaz Observador a ser implementada por las clases InformeMensual y GraficoGAA (Figura 4 y Listado 2):

Interfaz Observador

```
public interface Observador {
    public void actualizarDatos(Observable sujeto);
}
```

El GestorReportes hace uso de esta interfaz para notificar a sus observadores.

Figura 2 — Interfaz Observable y su Implementador

<<interfaz>> Observable	GestorReportes
notificarObservadores() registrar(obs:Observador) darDeBaja(obs:Observador)	getDepartamento():String setDepartamento(dept:String) notificarObservadores() registrar(obs:Observador) darDeBaja(obs:Observador) departamento:String listaObservadores:Vector

Listado 1 — Clase GestorReportes

```
public class GestorReportes extends JFrame
    implements Observable {
    ...
    private Vector listaObservadores;
    private String departamento;

    public GestorReportes() throws Exception {
        ...
        listaObservadores = new Vector();
        ...
    }

    public void registrar(Observador obs) {
        // Agregar a la lista de observadores
        listaObservadores.addElement(obs);
    }

    public void darDeBaja(Observador obs) {
        // Eliminar de la lista de observadores
    }

    public void notificarObservadores() {
        // Enviar notificación a todos los observadores
        for (int i = 0; i < listaObservadores.size(); i++) {
            Observador obs =
                (Observador) listaObservadores.elementAt(i);
            obs.actualizarDatos(this);
        }
    }

    public String getDepartamento() {
        return departamento;
    }

    public void setDepartamento(String dept) {
        departamento = dept;
    }

    // Clase interna para manejar eventos de botones
    class ManejadorBotones implements ActionListener {
        GestorReportes sujeto;

        public void actionPerformed(ActionEvent e) {
            if (e.getActionCommand().equals(GestorReportes.SALIR)) {
                System.exit(1);
            }
            if (e.getActionCommand().equals(GestorReportes.SET_OK)) {
                String dept = (String)
                    cmbListaDepartamentos.getSelectedItem();
                // cambio de estado
                sujeto.setDepartamento(dept);
                sujeto.notificarObservadores();
            }
        }

        public ManejadorBotones() {}

        public ManejadorBotones(GestorReportes gestor) {
            sujeto = gestor;
        }
    }
} // fin de la clase
```

Figura 3 — Interfaz de Usuario del GestorReportes

[Seleccionar Departamento: Muebles ▼ — Botones: OK | Salir]

Figura 4 — Jerarquía de la Interfaz Observador

<<interfaz>> Observador actualizarDatos(sujeto:Observable)	InformeMensual actualizarDatos(sujeto:Observable)	GraficoGAA actualizarDatos(sujeto:Observable)
---	---	---

Asociación Sujeto-Observador

Normalmente, un cliente primero crea una instancia del sujeto (GestorReportes). Cada vez que se crea un objeto Observador (por ejemplo, InformeMensual, GraficoGAA), este le pasa la referencia a la instancia del Sujeto como parte de la llamada a su constructor. El objeto Observador se registra a sí mismo con esta instancia del Sujeto:

Código del Cliente — Clase VistaAdministrador

```
public class VistaAdministrador {
    ...
    public static void main(String[] args) throws Exception {
        // Crear el Sujeto
        GestorReportes objSujeto = new GestorReportes();

        // Crear los Observadores (se registran automáticamente)
        new InformeMensual(objSujeto);
        new GraficoGAA(objSujeto);
    }
} // fin de la clase
```

Listado 2 — Clase InformeMensual como Observador

```
public class InformeMensual extends JFrame implements Observador {
    ...
    private GestorReportes objGestorReportes;

    public InformeMensual(GestorReportes inp_objGestorReportes)
        throws Exception {
        super("Patrón Observador - Ejemplo");
        objGestorReportes = inp_objGestorReportes;
        // Crear controles y etiquetas
        ...
        objGestorReportes.registrar(this);
    }

    public void actualizarDatos(Observable sujeto) {
        if (sujeto == objGestorReportes) {
            // Obtener el estado del sujeto
            String departamento = objGestorReportes.getDepartamento();
            lblTransacciones.setText(
                "Transacciones del Mes Actual - " + departamento);

            Vector listaTrn = getTransaccionesMesActual(departamento);
            String contenido = "";
            for (int i = 0; i < listaTrn.size(); i++) {
                contenido += listaTrn.elementAt(i).toString() + "\n";
            }
            taTransacciones.setText(contenido);
        }
    }
}
```

```

    }

    private Vector getTransaccionesMesActual(String departamento) {
        Vector v = new Vector();
        UtilArchivo util = new UtilArchivo();
        Vector todasFilas = util.archivoAVector("Transacciones.txt");

        // Mes actual
        Calendar cal = Calendar.getInstance();
        cal.setTime(new Date());
        int mes = cal.get(Calendar.MONTH) + 1;
        String busqueda = departamento + "," + mes + ",";

        int j = 1;
        for (int i = 0; i < todasFilas.size(); i++) {
            String str = (String) todasFilas.elementAt(i);
            if (str.indexOf(busqueda) > -1) {
                StringTokenizer st = new StringTokenizer(str, ",");
                st.nextToken(); // omitir el departamento
                str = " " + j + ". " + st.nextToken() + " " +
                    st.nextToken() + "~~~" +
                    st.nextToken() + " Artículos" + "~~~" +
                    st.nextToken() + " Dólares";
                j++;
                v.addElement(str);
            }
        }
        return v;
    }
} // fin de la clase

```

Flujo Lógico

1. Usando la interfaz de usuario del GestorReportes (Figura 3), cuando el usuario selecciona un departamento concreto y hace clic en el botón Aceptar, el GestorReportes experimenta un cambio en su estado interno (es decir, el valor de la variable de instancia departamento cambia).
2. En cuanto se establece el nuevo estado, el GestorReportes invoca el método actualizarDatos(Observable) sobre los objetos InformeMensual y GraficoGAA actualmente registrados.
3. Como parte del método actualizarDatos, ambos objetos de reporte:
 - a. Verifican que el sujeto que invocó el método actualizarDatos sea efectivamente la misma instancia del Sujeto con la que se registraron, para evitar que los observadores respondan a notificaciones no intencionadas.
 - b. Consultan al GestorReportes su estado actual mediante el método getDepartamento.
 - c. Recuperan los datos correspondientes del archivo de datos para su visualización (Figuras 5 y 6).

El diagrama de secuencia de la Figura 7 muestra la comunicación entre los distintos objetos durante la ejecución de la aplicación.

Cada vez que cambia la implementación de la lógica de cambio de estado del GestorReportes, ninguno de los observadores se verá afectado. Del mismo modo, al agregar un nuevo observador, no es necesario modificar la clase GestorReportes.

Figura 5 — Vista del InformeMensual

```
Transacciones del Mes Actual - Muebles12/10~~~~2 Artículos~~~~2000
Dólares12/15~~~~2 Artículos~~~~2000 Dólares12/1~~~~2 Artículos~~~~2000
Dólares
```

Figura 6 — Vista del GraficoGAA

```
Informe GAA - MueblesEne $1000 | Feb $5000 | Mar $4000 | Abr $1000May
$6000 | Jun $5000 | Jul $1000 | Ago $2000Sep $1000 | Oct $4000 | Nov
$1200 | Dic $6000
```

Figura 7 — Flujo de Mensajes de la Aplicación

```
:VistaAdministrador → crear() GestorReportes→ crear(obj:GestorReportes)
InformeMensual → registrar(objInformeMensual:InformeMensual)→
crear(obj:GestorReportes) GraficoGAA → registrar(objGraficoGAA:GraficoGAA) [al
seleccionar un departamento]→ actualizarDatos() → getDepartamento()→
actualizarDatos() → getDepartamento()
```

3. Preguntas de Práctica

4. Proporcione una implementación para el método darDeBaja de la clase GestorReportes.
5. En general, puede provocar distintos problemas el hecho de que un observador modifique el estado del sujeto (directa o indirectamente) mientras intenta actualizar su propio estado como parte de su método actualizarDatos(Observable). Piense en diferentes formas de manejar un escenario de este tipo.
6. Diseñe e implemente una aplicación para buscar en una base de datos de empleos que:
 - d. Permita al usuario seleccionar una habilidad de software específica y el número de años de experiencia.
 - e. Muestre en una ventana una lista de todas las ofertas de trabajo que requieren la habilidad y la experiencia seleccionadas, con sus detalles.
 - f. Muestre en una tercera ventana una lista de todos los candidatos con la habilidad y la experiencia seleccionadas.
7. Diseñe e implemente una aplicación para el monitoreo y reporte de eventos con la siguiente funcionalidad:
 - g. Cada vez que ocurra un evento, este se envía primero a un GestorEventos, que actúa como publicador.
 - h. Cada vez que el GestorEventos recibe un evento, lo almacena en una base de datos y envía notificaciones a los siguientes tres observadores para que realicen la acción necesaria:
 - i. Un objeto EnviadorAlertas que envía notificaciones (por correo electrónico, SMS, etc.) a diferentes usuarios según el evento ocurrido.
 - ii. Dos objetos de reporte que muestran los datos del evento en distintos formatos.